Servlet

Q. what is Servlet?

Servlet is a server side technology which is used for develop the dynamic web pages.
Servlet technology is used to create a web application (resides at server side and generates a dynamic web page).
Servlet technology is robust and scalable because of java language. Before Servlet, CGI (Common Gateway Interface) scripting language was common as a server-side programming language. However, there were many disadvantages to this technology.

Disadvantages of CGI

There are many problems in CGI technology:

- If the number of clients increases, it takes more time for sending the response.
- For each request, it starts a process, and the web server is limited to start processes.
- It uses platform dependent language e.g. C, C++, perl.

There are many advantages of Servlet over CGI. The web container creates threads for handling the multiple requests to the Servlet. Threads have many benefits over the Processes such as they share a common memory area, lightweight, cost of communication between the threads are low. The advantages of Servlet are as follows:

1. **Better performance:** because it creates a thread for each request, not process.
2. **Portability:** because it uses Java language.
3. **Robust:** JVM manages Servlets, so we don't need to worry about the memory leak, garbage collection, etc.
4. **Secure:** because it uses java language.

Q. what is server side technology?

Server side means those technology need server to execute its application.

Q. what is dynamic web pages?

Dynamic web pages means those pages connected with database at server side called as dynamic web pages and those pages can change its content at program run time called as dynamic web pages.

Q. what is server?

Server is application or it is software which is used for manage the resource or application at central location and accept the client request and provide appropriate response to client as per the request of client called as server.

Q. what is client?

Client is also an application which design for send request to server application and get response from a server.
Note: In the case of web, browser work as client software.

Q. How client and server communicate with each other in web?

Client and server communicate in network with each other with the help of protocol.

Q. what is protocol?
Protocol means standard rules and regulation of communication between two parties called as protocol.

Types of protocol

TCP/IP: Transmission Control protocol normally we use this protocol in LAN and it is used for secure data transmission such web browsing, email, file transfer etc and it is used for send data using packets

http: hyper text transfer protocol it is specially design for web application this protocol normally send request using text/html format and get response from server in text/html form

https: hyper text transfer protocol security layer and it used for web application this protocol normally send request using text/html format and get response from server in text/html form with security feature.

POP : Post Office Protocol it work as email client protocol means it is used for read email messages send by server and download it on local machine.

SMTP: Simple mail transfer protocol it is specially design sending and receiving mail
Etc.

Q. what is static web pages?

static pages means those pages not connected with database and cannot modify its content at run time called as static web pages.

Servlet Types

The two main servlet types are, generic and HTTP.

- Generic servlets:
  - Extend `javax.servlet.GenericServlet`.
  - Contain no inherent HTTP support or any other transport protocol, so that protocol is independent.
- HTTP servlets:
  - Extend `javax.servlet.HttpServlet`.
  - Contain built-in HTTP protocol support and are more useful in a Sun Java System Web Server 7.0 environment.

For both servlet types, you must implement the initializer method `init()` to allocate and initialize the servlet, and the destructor method `destroy()` to deallocate resources.

All servlets must implement a `service()` method, which is responsible for handling servlet requests. For generic servlets, override the service method to provide routines for handling requests. HTTP servlets provide a service method that automatically routes the request to another method in the servlet based on which HTTP transfer method is used. For HTTP servlets, override `doPost()` to process POST requests, `doGet()` to process GET requests, and so on.

How to Create Application using a Servlet

If we want to create application using a servlet we have some important steps.

1) Download and installed apache tomcat as web server

Apache tomcat is application and it work as web server
https://tomcat.apache.org/download-90.cgi

When we installed the server we need to give port number

Q. what is port number?

Port number is unique identity of server in machine and by using port number
We can access server at client side means port number is important because if we think
Computer single machine can have more than one server and if we want to access
Particular server we provide unique number to server called as port number.

2) Set CATALINA_HOME path

Here CATALINA is tomcat means if we say CATALINA_HOME means we need to path of tomcat root directory

Steps to configure the CATALINA_HOME

Open search menu --- search advanced system setting ---- click on environmental variable ---- select system variable ----- click on new button ------ variable name=CATALINA_HOME and
variable value = path of tomcat root directory (C:\Program Files\Apache Software Foundation\Tomcat 9.0) --- click on ok

3) Set servlet-api.jar file path

Q. what is servlet-api.jar?

---

servlet-api.jar is external file present in tomcat\lib or present in any server which contain the all classes and interfaces required for Servlet application.
Servlet packages, classes and interfaces not present in JDK setup it is external api name which contain all classes and interface required for servlet known as servlet-api.jar
If we want to use any .jar file by using notepad in your system we need to set its class path

How to set the class path of servlet-api.jar file

If we want to set the class path of any .jar file we have following steps.

Start menu --- advanced system setting ---- environmental variable ---- system variable --- variable name =CLASSPATH and variable value= tomcat\lib\servlet-api.jar need to give complete path like as
Example: C:\Program Files\Apache Software Foundation\Tomcat 9.0\lib\ servlet-api.jar

4) Write Simple Servlet code

If we want to work with a servlet we need to write minimum following code.

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class DemoServ extends HttpServlet
{  public void doGet(HttpServletRequest request, HttpServletResponse response)throws
ServletException, IOException
  {       response.setContentType("text/html");
           PrintWriter out=response.getWriter();
           out.println("Welcome in servlet");
  }
}
```

import javax.servlet.*;
import javax.servlet.http.*;

These packages contain all classes and interfaces required for servlet application so we need to import these packages in your application.

public class DemoServ extends HttpServlet

HttpServlet is abstract class from javax.servlet.http package and it is used for to create web page using servlet so if we want to create any web page using servlet we need to create user defined class and inherit the HttpServlet class in it

Note: means here we can say DemoServ is web page created by servlet.

Internally HttpServlet is child of GenericServlet and GenericServlet is implementer class of Servlet interface.

Internal code

```
interface Servlet{
    public abstract void init(ServletConfig);
    public ServletConfig getServletConfig();
    public void service(ServletRequest, ServletResponse);
    public void destroy();
    public String getServletInfo();
}
public abstract class GenericServlet implements Servlet{
          public String getInitParameter(String paramName)
          public ServletContext getServletContext()
          public void init()
          public void log()
}
```

HttpServlet

The HttpServlet class extends the GenericServlet class and implements Serializable interface. It provides http specific methods such as doGet, doPost, doHead, doTrace etc.

```
Public abstract class HttpServlet extends javax.servlet.GenericServlet{

     public void doGet(HttpServletRequest request, HttpServletResponse response)throws
ServletException, IOException
      public void doPost(HttpServletRequest request, HttpServletResponse response)throws
ServletException, IOException
     public void doHead(HttpServletRequest request, HttpServletResponse response)throws
ServletException, IOException
```

public void doDelete(HttpServletRequest request,HttpServletResponse response)throws ServletException,IOException

public void doPut(HttpServletRequest request,HttpServletResponse response)throws ServletException,IOException

public void doTrace(HttpServletRequest request,HttpServletResponse response)throws ServletException,IOException

}

public void doGet/doPost(HttpServletRequest request, HttpServletResponse response)throws ServletException, IOException

---

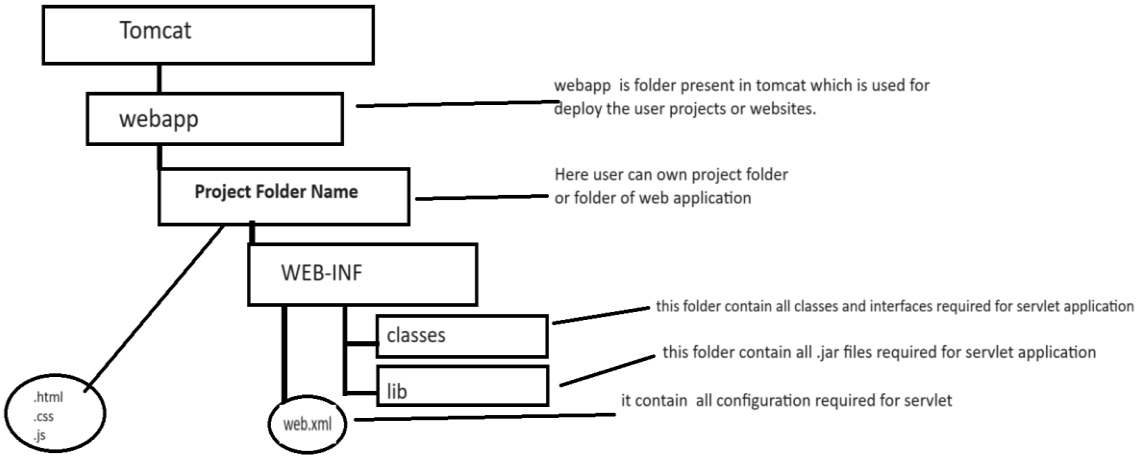
This method is used for accept the client request by using HttpServletRequest interface and provide response to client using HttpServletResponse interface

response.setContentType("text/html"): this method is used for send response to client using text/html format by default and you can send to response to client using other format like as json, word etc

PrintWriter out=response.getWriter(): Here getWriter() is method of HttpServletResponse and this method return reference of PrintWriter from java.io package and PrintWriter is class which is used for generate response and show on web page to end user.

Once we create servlet code you can create standard folder structure provided by Servlet

5) Create Standard folder structure set by apache



Once we create folder structure you can save your servlet code in classes folder present in WEB-INF and compile servlet program

6) Write web.xml file and configure servlet

---

web.xml file is deployment descriptor it contain all configuration required for servlet means using this file we can create object of servlet class and can configure URL of servlet etc.

If we want to configure Servlet using web.xml file we have to write following tags in it.

```
 <servlet>
   <servlet-name>name of servlet</servlet-name>
  <servlet-class> classname of servlet</servlet-class>
</servlet>
<servlet-mapping>
   <servlet-name>name of servlet</servlet-name>
   <url-pattern>/urlname</url-pattern>
</servlet-mapping>
```

<servlet-name>name of servlet</servlet-name>: servlet-name work as reference of Servlet class.

<servlet-class> classname of servlet</servlet-class>: servlet-class indicates the class name of servlet whose object created by servlet container

<url-pattern>/urlname</url-pattern>: url-pattern means here we specify the URL name or hyperlink for access a servlet by end user.

Complete source code of web.xml file

```
<?xml version="1.0" encoding="UTF-8"?>

<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
                http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
 version="4.0"
 metadata-complete="true">
  <servlet>
         <servlet-name>d</servlet-name>
         <servlet-class>DemoServ</servlet-class>
  </servlet>
  <servlet-mapping>
          <servlet-name>d</servlet-name>
          <url-pattern>/test</url-pattern>
  </servlet-mapping>
</web-app>
```

7) Deploy the servlet application

Open the web browser and type the following type of URL

Syntax: http://localhost:portnumber/projectname/urlname

Example: http://localhost:9090/first/test

Following screenshot shows the working of servlet

How to Create Servlet Application by using eclipse

Steps to create Servlet Application by using eclipse

1) Open eclipse

2) Create Dynamic web project

Steps to create dynamic web project

file ---- new --- other --- web----- dynamic web project --- click on next button ---- give project name ------ Click on next --- next and click on finish button

3) Create user defined package under src/main/java

Right click on src/main/java ---- select new option ---- select package --- give packagename and click on finish.

4) Create servlet under package

right click on package ---- select new option --- select other ---- select web --- select servlet -- give servlet class name ---- click on next and configure URL of servlet ---- click on next and finish.

Once we create servlet we get following code with compile time error for package javax.servlet. and javax.servlet.http shown in following diagram.


```
1  package org.techhub;
2  import java.io.IOException;
8  @WebServlet("/test")
9  public class TestServ extends HttpServlet {
10     protected void doGet(HttpServletRequest request, HttpServletResponse response)
11             throws ServletException, IOException {
12         response.getWriter().append("Served at: ").append(request.getContextPath());
13
14     }
15     protected void doPost(HttpServletRequest request, HttpServletResponse response)
16             throws ServletException, IOException {
17         // TODO Auto-generated method stub
18         doGet(request, response);
19     }
20 }
```

If we want to resolve the compile time error we need to add servlet-api.jar file in your lib folder of application after this you can write your code in doGet() or doPost() method

```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/test")
public class TestServ extends HttpServlet {
        protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
                response.setContentType("text/html");
                PrintWriter out=response.getWriter();
                out.println("<h1>good morning india</h1>");
        }
        protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
                doGet(request, response);
        }
}
```

4) Deploy application on server or Run application on Server

Right click on servlet page ---- select run as option --- select run on server --- select apache server and its suitable version which we installed in your machine ----click on next--- click on browse button --- select root folder of tomcat directory where it is installed --- click on next and finish button

Output:

Example: we want to declare two variables using a servlet and calculate its addition

```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/first")
public class Demo extends HttpServlet {
	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
				response.setContentType("text/html");
				PrintWriter out=response.getWriter();
				int a, b, c;
				a=100;
				b=200;
				c=a+b;
				out.println("<h1>Addition is "+c+"</h1>");
	}
	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
		doGet(request, response);
	}
}
```

Output

If we think about above code we calculate the addition of two fix values we want to accept input from keyboard and then calculate its addition.

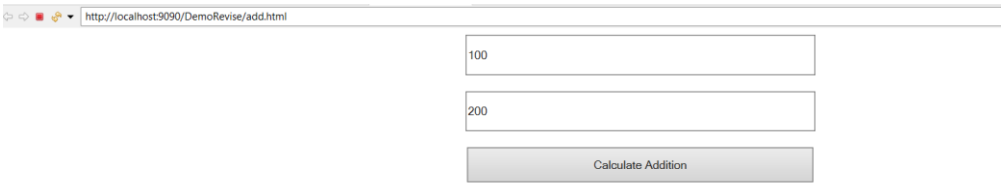
If we want to accept the input in web application we need to use HTML form

How to submit HTML Form to servlet

If we want to submit HTML form to servlet we have following steps.

1) Create Dynamic web project

2) Create HTML page under web app folder and design form



add.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
  input{
    width: 400px;
    height: 40px;
  }
</style>
</head>
<body>
  <center>
    <input type='text' name='first' value=''/><br/><br/>
    <input type='text' name='second' value=''/><br/><br/>
    <input type='submit' name='s' value='Calculate Addition'/>
  </center>
</body>
</html>
```

3) Use Form Tag in following fashion and submit to servlet as request

---

form tag is used for submit form data to server as request

Syntax of form tag

<form name="formname" action="url" method="GET/POST" enctype="application/x-www-urlencoded or multipart/form-data">
</form>

Description about form tag

---

name=”formname”: this attribute indicate the name of form
action=”url”: this attribute set the url of servlet where we want to submit data or
Server side where we send request as form data
method=”GET/POST”: Method indicate how we submit form data to server
There are two ways to submit form data

a) Using GET method: when we use GET method then all form data visible in browser address bar in the form of name and value pair

b) Using POST method: when we use POST method then form data send to server by using body of the page.

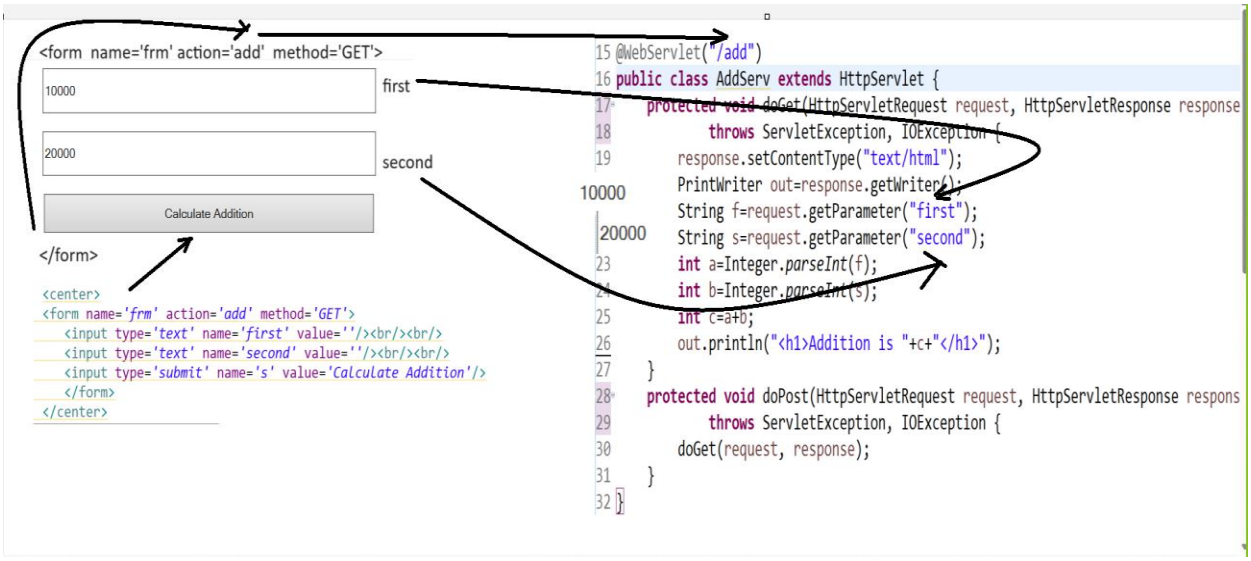
Note: we will discuss enctype in the file uploading topic.

4) Create Servlet and accept request from form and read its data at server side

---

If we want to accept the form data at server side we have HttpServletRequest interface as parameter in doGet() or in doPost() method and this interface provide one method to us name as getParameter() and this method help us to accept the form data request means this method accept form control name and return its value and if form control name is wrong or not present then return null.

String getParameter(String formcontrolname): this method accept form data and return its value.



Complete source code of above example

add.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
```

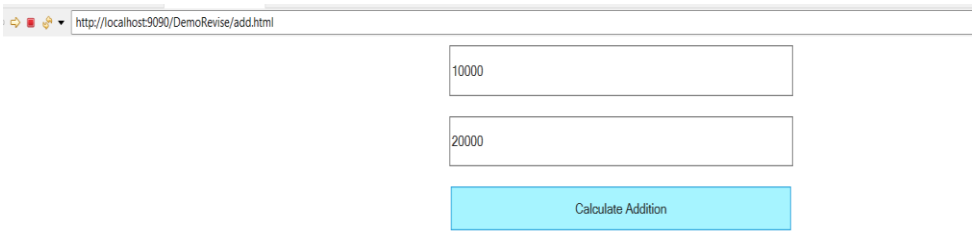
```
<title>Insert title here</title>
<style>
  input{
    width:400px;
    height: 40px;
  }
</style>
</head>
<body>
   <center>
   <form name='frm' action='add' method='GET'>
     <input type='text' name='first' value=''/><br/><br/>
     <input type='text' name='second' value=''/><br/><br/>
     <input type='submit' name='s' value='Calculate Addition'/>
     </form>
   </center>
</body>
</html>
```

AddServ.java

```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/add")
public class AddServ extends HttpServlet {
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
                        throws ServletException, IOException {
                response.setContentType("text/html");
                PrintWriter out=response.getWriter();
                String f=request.getParameter("first");
                String s=request.getParameter("second");
                int a=Integer.parseInt(f);
                int b=Integer.parseInt(s);
                int c=a+b;
                out.println("<h1>Addition is "+c+"</h1>");
        }
        protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
                    throws ServletException, IOException {
                doGet(request, response);
        }
}
```

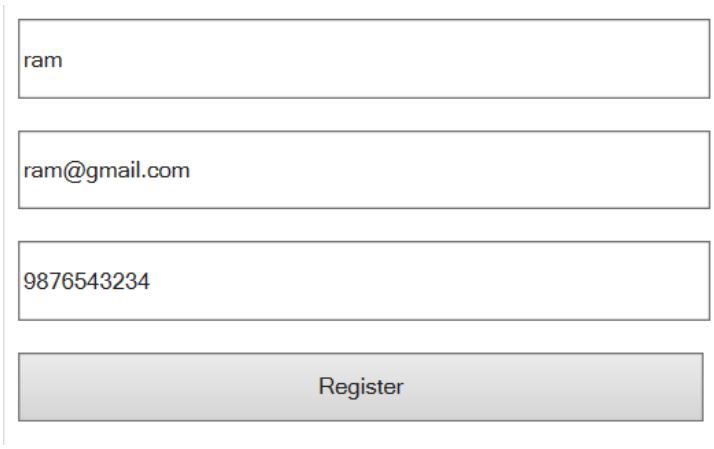
Output:



Example: We want to design registration page with field name email and contact and store database table.

Steps to develop above application

1. Create Dynamic web project.
2. Add mysqlconnector.jar file in lib folder of application
3. Design html form and send request to server using form tag



style.css

```
@charset "ISO-8859-1";
.control{
        width:400px;
        height:40px;
}
```

register.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<link rel="stylesheet" href="CSS/style.css"/>
</head>
<body>
<form name='frm' action='reg' method='GET'>
  <input type='text' name='name' value='' class="control" /><br/><br/>
  <input type='text' name='email' value='' class="control" /><br/><br/>
  <input type='text' name='contact' value='' class="control" /><br/><br/>
  <input type='submit' name='s' value='Register' class="control" /><br/><br/>
  </form>
</body>
</html>
```

4) Write Servlet code and accept form data and store in database using JDBC

```
package org.techhub;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.sql.*;
@WebServlet("/reg")
public class RegisterServ extends HttpServlet {
	private static final long serialVersionUID = 1L;
	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
			response.setContentType("text/html");
			PrintWriter out=response.getWriter();
			String n=req-uest.getParameter("name");
			String e=request.getParameter("email");
			String c=request.getParameter("contact");
			try {
				Class.forName("com.mysql.cj.jdbc.Driver");
			Connection
conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/mysql","root","root");
```

```
                        if(conn!=null) {
                         PreparedStatement stmt=conn.prepareStatement("insert into julyservreg
values('0',?,?,?)");
                         stmt.setString(1, n);
                         stmt.setString(2, e);
                         stmt.setString(3, c);
                         int value=stmt.executeUpdate();
                         if(value>0) {
                                  out.println("<h1>Registration Success....</h1>");
                         }
                         else {
                                  out.println("<h1>Registration Failed.....</h1>");
                         }
                        }
                        else {
                                 out.println("Database is not connected");
                        }
                }
                catch(Exception ex) {
                          System.out.println("Error is "+ex);
                }

        }

        protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

                doGet(request, response);
        }

}
```

RequestDispatcher interface

RequestDispatcher is used for forward the page from servlet and include another page content in servlet.

The RequestDispatcher interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp. This interface can also be used to include the content of another resource also. It is one of the ways of servlet collaboration.

There are two methods defined in the RequestDispatcher interface.

Steps to work with RequestDispatcher interface

---

a) add javax.servlet.* and javax.servlet.http.* package in application

---

```
import javax.servlet.http.*;
import javax.servlet.*;
```

b) Create reference of RequestDispatcher

If we want to create reference of RequestDispatcher we have method name as getRequestDispatcher() and this is the member of HttpServletRequest interface.

Syntax: RequestDispacher r=request.getRequestDispatcher(String urlanme);

```
import  javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class ValidServ extends HttpServlet
{
   public void doGet(HttpServletRequest request,HttpServletResponse response)throws ServletException,IOException
  {
      response.setContentType("text/html");
      PrintWriter out=response.getWriter();
       RequestDispatcher r=request.getRequestDispatcher("welcome");//welcome is url of destination servlet
  }
}
```

c) call its forward() and include() method

---

void forward(HttpServletRequest, HttpServletResponse): Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

void include(HttpServletRequest, HttpServletResponse): Includes the content of a resource (servlet, JSP page, or HTML file) in the response.

```
import  javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class ValidServ extends HttpServlet
{
   public void doGet(HttpServletRequest request,HttpServletResponse response)throws ServletException,IOException
  {
      response.setContentType("text/html");
      PrintWriter out=response.getWriter();
       RequestDispatcher r=request.getRequestDispatcher("welcome");//welcome is url of destination servlet
        r. forward(request,response);


  }
}
```

Complete source code of above diagram

style.css

```
@charset "ISO-8859-1";

#inputCtrl{
        width:400px;
        height:40px;
        border-radius:10px;
}
```

login.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<link rel="stylesheet" href="CSS/style.css"/>
</head>
<body>
<form name='frm' action='valid' method='GET'>
 <input type='text' name='user' value='' id="inputCtrl"/><br/><br/>
 <input type='text' name='pass' value='' id="inputCtrl"/><br/><br/>
 <input type='submit' name='s' value='Login'  id="inputCtrl"/><br/><br/>
 </form>
</body>
</html>
```

ValidateServ.java

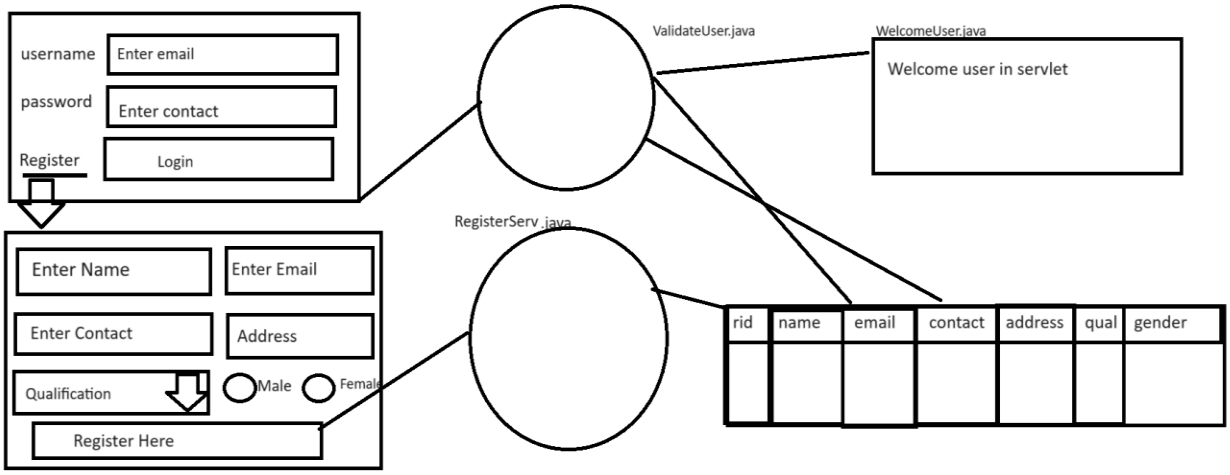
```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/valid")
public class ValidateServ extends HttpServlet {
	protected void doGet(HttpServletRequest request, HttpServletResponse response)
			throws ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		String username=request.getParameter("user");
		String password=request.getParameter("pass");
		if(username.equals("admin") && password.equals("admin")) {
			RequestDispatcher r=request.getRequestDispatcher("welcome");
			r.forward(request, response);
		}
		else {
			RequestDispatcher r=request.getRequestDispatcher("login.html");
			r.include(request, response);
			out.println("<h1>Invalid username and password</h1>");
	 }
	}

}
```

WelcomePage.java

```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/welcome")
```

```
public class WelcomePage extends HttpServlet {
        protected void doGet(HttpServletRequest request, HttpServletResponse response)
                        throws ServletException, IOException {
          response.setContentType("text/html");
          PrintWriter out=response.getWriter();
          out.println("<h1>I am welcome page</h1>");
        }
}
```

Assignment for homework



Technologies

1. Front End: HTML/CSS/Bootstrap/JS

2. Backend: Core JAVA, JDBC, Servlet

3. Database: MYSQL

Note: same user should not be register repeat means email and contact should not be duplicated in database when we submit form and if email and contact exist in database when we click on register button then application should show the message user already exist and if email and contact not present in database then application should show message registration success.

Session Management

Session Tracking is a way to maintain state (data) of a user. It is also known as **session management** in servlet.

Q. what is session?

Session is a communication period between client and server over the network called as session.

Example: if we login for particular website then time period between login to logout called as session.

Q. why use session or what is benefit of session?

---

1. Session helps us to maintain privacy of client data:
Example: suppose consider we are working online exam portal and we provide login to every student and if student login then student should display only his own result not others
2. Session help us to track user activity
3. Session help us to store user data temporary at client side whenever client is login
4. Session help us to count the current login users on web site.
5. Session help us to provide security to user data
Note: http is stateless protocol means when client send more than one request to server then server consider new client every time called as stateless protocol so when we use session then server create one session id and send to client and when client to revisit or resend request to server then server verify its session id if server not found session id then consider it is new client and if server found existing session id then consider it is old client.
Note: we can say when we use HttpSession interface then your http protocol converts from stateless to stateful protocol.
Http protocol is a stateless so we need to maintain state using session tracking techniques. Each time user requests to the server, server treats the request as the new request. So we need to maintain the state of a user to recognize to particular user.

Why use Session Tracking?

Session tracking is used to recognize the particular user.

How to implement the session practically?

If we want to implement the session practically using a servlet we have following ways.

1) Using HttpSession interface
2) Using Cookies
3) Using URL Rewriting
4) Using hidden field

How to use the session using HttpSession interface

If we want to work with HttpSession interface we have following steps.

1) add javax.servlet.* and javax.servlet.http.* package in application.

```
import javax.servlet.*;
import javax.servlet.http.*;
```

2) Create reference of HttpSession interface.

If we want to create reference of HttpSession interface we have getSession() method of HttpServletRequest interface.

Syntax: HttpSession session=requesterf.getSession(boolean): this method create new session if we pass true value in it otherwise use existing session.

or

HttpSession session=requestref.getSession() : this method is used for use existing session created by server.

1. **public HttpSession getSession():**Returns the current session associated with this request, or if the request does not have a session, creates one.
2. **public HttpSession getSession(boolean create):** Returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session.

3) Use its methods to Work with HttpSession.

HttpSession interface provide some inbuilt method to us

String getId(): this method return session id generated by server for every client

void setAttribute(Object key, Object value): this method is used for store data in session in the form of key and value pair.

Object getAttribute(Object key): this method can return data from session using its key and if key not found return null

boolean isNew(): this method can check session is new or not if new return true otherwise return false.

void invalidate(): this method can destroy the session.

Etc.

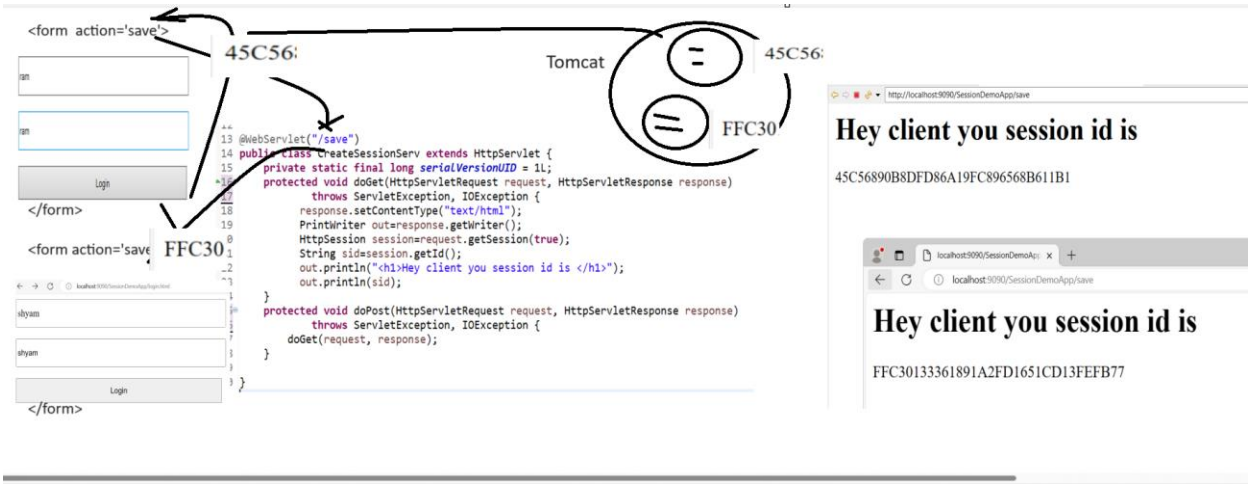
login.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
 input{
  width:400px;
  height:40px;
 }
</style>
</head>
```

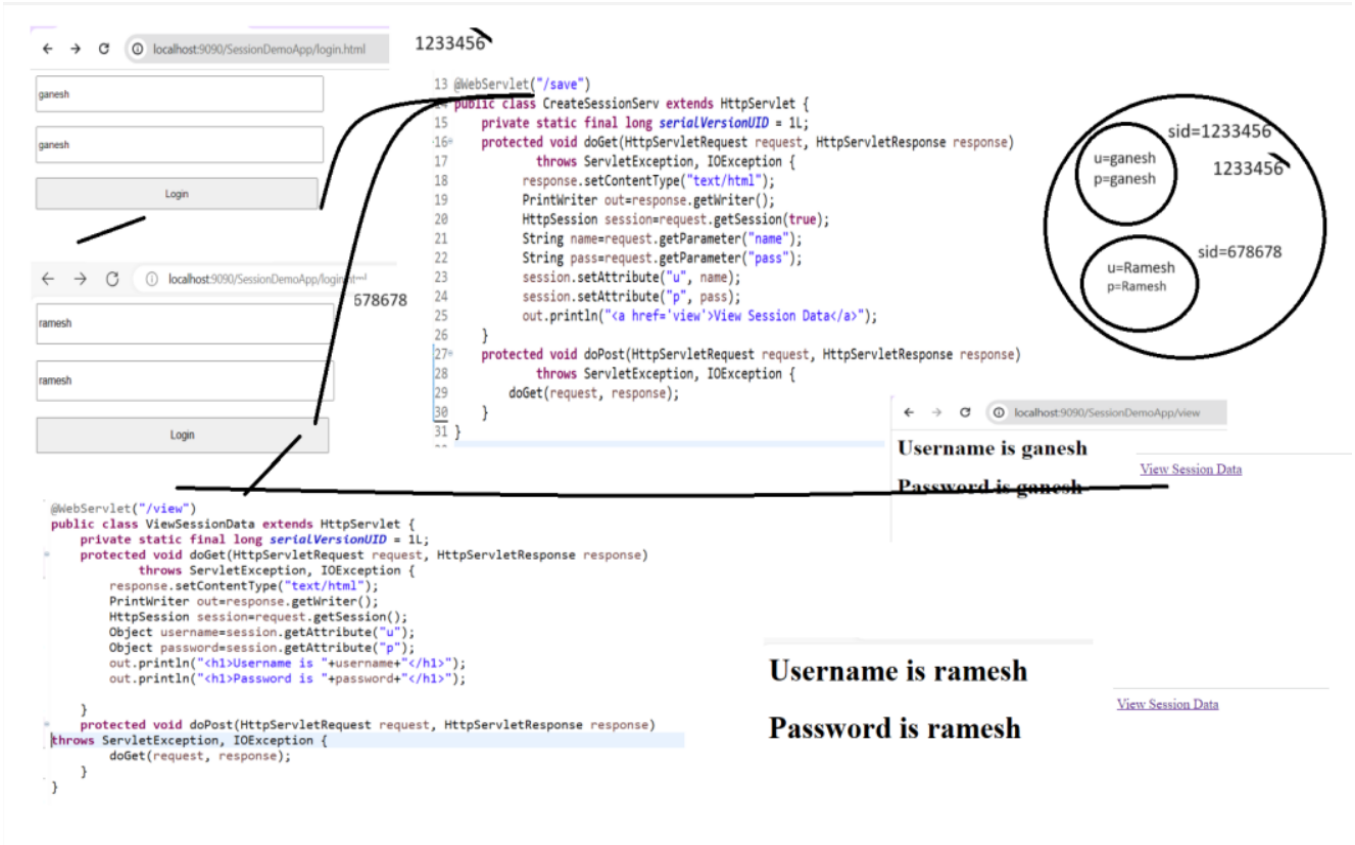
```
<body>
<form name='frm' action='save' method='POST'>
 <input type='text' name='name' value=''/><br/><br/>
 <input type='text' name='pass' value=''/><br/><br/>
 <input type='submit' name='s' value='Login'/>
 </form>
</body>
</html>
```

Servlet code

```
package org.techhub;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.*;
@WebServlet("/save")
public class CreateSessionServ extends HttpServlet {
	private static final long serialVersionUID = 1L;
	protected void doGet(HttpServletRequest request, HttpServletResponse response)
				throws ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		HttpSession session=request.getSession(true);
		String sid=session.getId();
		out.println("<h1>Hey client you session id is </h1>");
		out.println(sid);
	}
	protected void doPost(HttpServletRequest request, HttpServletResponse response)
				throws ServletException, IOException {
		doGet(request, response);
	}
}
```



Example: WAP to create login page and store its data in session object and display data on another servlet.



Source code

login.html

<!DOCTYPE html>

<html>

<head>

<meta charset="*ISO-8859-1*">

```
<title>Insert title here</title>
<style>
 input{
  width:400px;
  height:40px;
 }
</style>
</head>
<body>
<form name='frm' action='save' method='POST'>
 <input type='text' name='name' value=''/><br/><br/>
 <input type='text' name='pass' value=''/><br/><br/>
 <input type='submit' name='s' value='Login'/>
 </form>
</body>
</html>
```

CreateSessionServ.java

```
package org.techhub;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.*;
@WebServlet("/save")
public class CreateSessionServ extends HttpServlet {
	private static final long serialVersionUID = 1L;
	protected void doGet(HttpServletRequest request, HttpServletResponse response)
				throws ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		HttpSession session=request.getSession(true);
		String name=request.getParameter("name");
		String pass=request.getParameter("pass");
		session.setAttribute("u", name);
		session.setAttribute("p", pass);
		out.println("<a href='view'>View Session Data</a>");
	}
	protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
                throws ServletException, IOException {
        doGet(request, response);
    }
}
```

ViewSessionData.java

```
package org.techhub;
import java.io.*;
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/view")
public class ViewSessionData extends HttpServlet {
    private static final long serialVersionUID = 1L;
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
                throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out=response.getWriter();
        HttpSession session=request.getSession();
        Object username=session.getAttribute("u");
        Object password=session.getAttribute("p");
        out.println("<h1>Username is "+username+"</h1>");
        out.println("<h1>Password is "+password+"</h1>");
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
        doGet(request, response);
    }
}
```

Cookie

Cookie is part of session management Means we can handle the session by using cookie object in servlet.

A **cookie** is a small piece of information that is persisted between the multiple client requests.

A cookie has a name, a single value, and optional attributes such as a comment, path and domain qualifiers, a maximum age, and a version number.

Q. what is diff between Cookie and HttpSession?

---

1. HttpSession vanish its data when browser get closed and Cookie can store its data for longer time if your browser get closed.
2. HttpSession is object created at server and stores its data at server side and Cookie is object created at server side and stores its data at client side.

Note: Cookie may be responsible for leak user data so this is major reason when we store data in cookie try to avoid confidential data or encrypt at server and then store in cookie
If user disables the cookie at browser side or at client side then cookie works as HttpSession.

There are two types of Cookie

Temporary Cookie and Persistent cookie

1. Temporary Cookie: without age cookie called as temporary cookie or cookie vanishes its data when we close the browser called as temporary cookie.
2. Persistent Cookie: persistent cookie means cookie can persist or save its data for longer time if we close the browser means when we set age of cookie called as persistent cookie.

Non-persistent cookie

It is **valid for single session** only. It is removed each time when user closes the browser.

Persistent cookie

It is **valid for multiple sessions** it is not removed each time when user closes the browser. It is removed only if user logout or signout.

Advantage of Cookies

1. Simplest technique of maintaining the state.
2. Cookies are maintained at client side.

Disadvantage of Cookies

1. It will not work if cookie is disabled from the browser.
2. Only textual information can be set in Cookie object.

We have following constructors to create Cookie object

Cookie()	Constructs a cookie.
Cookie(String name, String value)	constructs a cookie with a specified name and value

How to works with Cookie

---

1. Add javax.servlet.* and javax.servlet.http.* package in application.

```
import javax.servlet.*;
import javax.servlet.http.*;
```

2. Create Object of Cookie class

If we want to create Cookie object we have to use following syntax
Cookie is class present is javax.servlet.http.* package.

Syntax: Cookie ref = new Cookie(String name/key, String value): this object is used for store data in the form of name and value pair so name is like as key

Sample code

```
Cookie c = new Cookie("u", "Ganesh");
```

3. Set its age: age decide the life span of cookie at client side **and** if we want to set age of cookie then we have method name as
void setMaxAge(int seconds): this method can set age of cookie in seconds.

Example:

```
Cookie c = new Cookie ("u", "Ganesh");
 c.setMaxAge(60*60*24*365);
```

4. Send cookie object as response to client from a server

If we want to send cookie object from server to client we have method name as
void addCookie(Cookie): this method is used for send cookie data from server to client

once we send the cookie as response to client and when client revisit to server then server need to read its cookie data if cookie object found in request then server consider it is existing client and if cookie not found in client request then server consider it is new client.

Read Cookie objects at server side.

If we want to read cookie object at server side we have method name as getCookies() and this method return Cookie array of objects.

Syntax:

```
Cookie ref[]= request.getCookies();
```

Sample code

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class ReadCookie extends HttpServlet
{
    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException{
            response.setContentType(“text/html”);
            PrintWriter out=response.getWriter();
            Cookie c[]=request.getCookies();
  }
}
```

String getName(): this method can return name of cookie or key of cookie which we set when we create object of cookie

String getValue(): this method can return actual data from cookie object

Example: we want to create login page and input username and password in login and send to server and store data in cookie object with life span 3 months and we want to create one more server name as ViewCookie where we want to read cookie data and display it

login.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
```

```
 input{
   width:400px;
   height:40px;
 }
</style>
</head>
<body>
<form name='frm' action='save' method='POST'>
  <input type='text' name='name' value=''/><br/><br/>
  <input type='password' name='pass' value=''/><br/><br/>
  <input type='submit' name='s' value='Login'/>
 </form>
  </body>
</html>
```

SaveCookie.java

```
package org.techhub;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/save")
public class SaveCookie extends HttpServlet {

	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		String username=request.getParameter("name");
		String password=request.getParameter("pass");
		Cookie c = new Cookie("n", username);
		c.setMaxAge(60*60*24*90);//3 month
		response.addCookie(c); //send cookie as response to client

		Cookie c1 = new Cookie("p", password);
		c1.setMaxAge(60*60*24*90);
```

```
                response.addCookie(c1);

                out.println("<a href='viewcookie'>View Cookie Data</a>");

        }
  protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
                doGet(request, response);
        }

}
```

ViewCookie.java

```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/viewcookie")
public class ViewCookie extends HttpServlet {

        protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
                response.setContentType("text/html");
                PrintWriter out=response.getWriter();
                Cookie c[]=request.getCookies();
           for(int i=0; i<c.length;i++) {
                 out.println(c[i].getName()+"   "+c[i].getValue()+"<br>");
           }
        }
        protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
                doGet(request, response);
        }
}
```

URL Rewriting

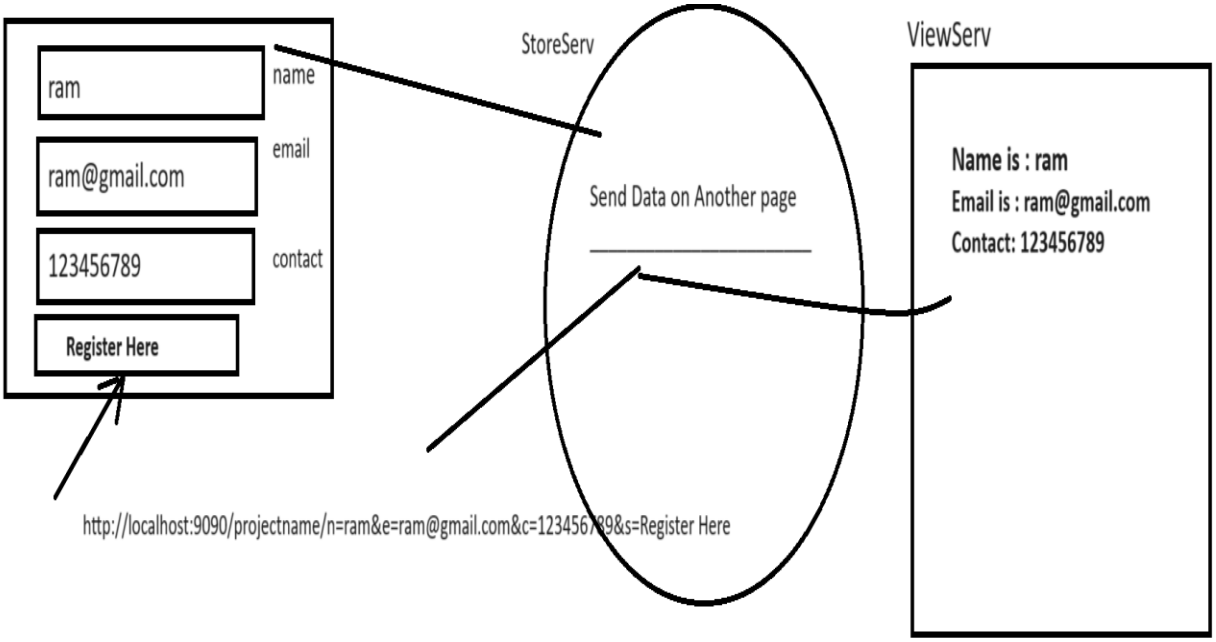
URL rewriting means we can send data as request through the hyperlink from one page to another page in the form name and value pair called as URL Rewriting.
If we want to work with URL Rewriting we have following syntax

Syntax:
http://localhost:portnumber/projectname/urlname?name=value&name=value&name=value

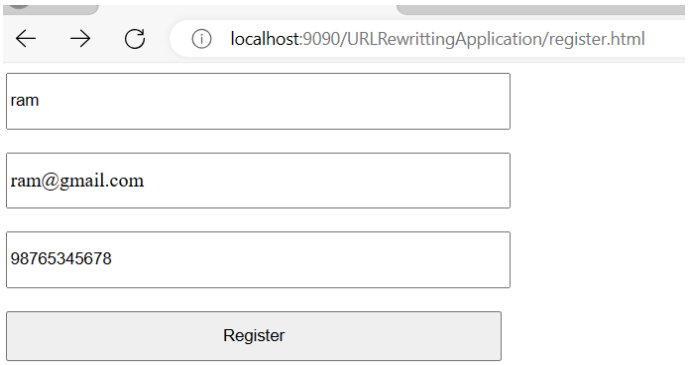
Note: here we pass request parameter in the form of name and value pair so when we use URL Rewriting then we need to use doGet() method for accept request send by URL means in the case of URL rewriting we cannot use doPost() method
So we want to accept the request send by URL we have method name as request.getParameter()

Syntax: String variablename=request.getParameter(String paramName);

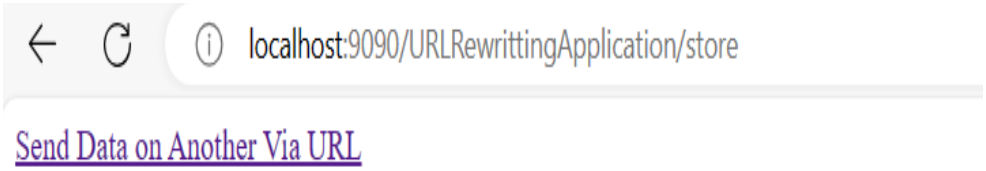
Example: we want to design registration page with name email and contact field and submit to servlet and accept its request and store in hyperlink and when we click on that hyperlink then we want to display the hyperlink stored data on designation page shown in following diagram.

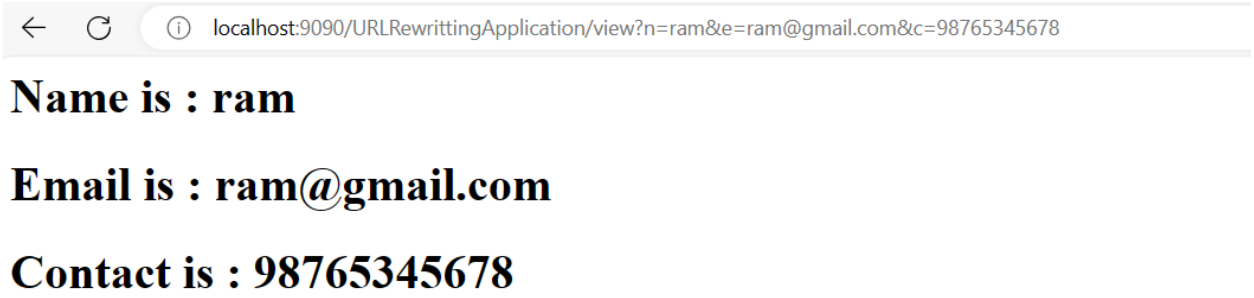


Output:



View page





Source code

login.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Insert title here</title>
<style>
 input{
   width:400px;
   height:40px;
 }
</style>
</head>
<body>
  <form name='frm' action='store' method='POST'>
```

```
   <input type='text' name='name' value=''/><br/><br/>
  <input type='text' name='email' value=''/><br/><br/>
  <input type='text' name='contact' value=''/><br/><br/>
  <input type='submit' name='s' value='Register'/>
  </form>
</body>
</html>
```

StoreServ.java

```
package org.techhub;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/store")
public class StoreServ extends HttpServlet {

	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
				response.setContentType("text/html");
				PrintWriter out=response.getWriter();
				String name=request.getParameter("name");
				String email=request.getParameter("email");
				String contact=request.getParameter("contact");
out.println("<a href='view?n="+name+"&e="+email+"&c="+contact+"'>Send Data on Another Via
URL</a>");
	}
	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		doGet(request, response);
	}

}
```

ViewUrlData.java

```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/view")
public class ViewUrlData extends HttpServlet {
	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		String name=request.getParameter("n");
		String email=request.getParameter("e");
		String contact=request.getParameter("c");
		out.println("<h1>Name is     : "+name+"</h1>");
		out.println("<h1>Email is    : "+email+"</h1>");
		out.println("<h1>Contact is : "+contact+"</h1>");
	}
	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		doGet(request, response);
	}
}
```

ServletContext and ServletConfig

ServletContext is object created at server side and it is known as application object means servlet context object created for website or application means suppose consider we have tomcat server and we deploy 10 projects or 10 websites on tomcat then we have 10 different context object created at server side .

If we think about session it is created for each user means if we have website and if 100 people login to website we have 100 sessions running on web site and session data is separate for every client but context or application object data is common to all clients of web application.

An object of ServletContext is created by the web container at time of deploying the project. This object can be used to get configuration information from web.xml file. There is only one ServletContext object per web application.

If any information is shared to many servlet, it is better to provide it from the web.xml file using the **<context-param>**element

Q. Why use ServletContext?

There are two reasons.
1. Configure data in web.xml file and access in servlet
2. Store data which we want to store at server side whenever server is running

Note: if we store data in context object or application object then data store at server side whenever server is running means application or context object data vanish when we restart server or stop server.

Q. Why we need to configure data in web.xml file and access in servlet?

Sometime if we have some hard coded data in application and if we want to change data in future then we need to change the data in servlet and again recompile servlet and deploy on server but it not possible every time in real time scenario or very tedious task so better way you can configure data in web.xml file and access in servlet via ServletContext object So the benefit is when we want to change data in future just need to change in web.xml file so changes automatically in servlet.

How to get the object of ServletContext interface

1. **getServletContext() method** of ServletConfig interface returns the object of ServletContext.
2. **getServletContext() method** of GenericServlet class returns the object of ServletContext.

Commonly used methods of ServletContext interface

1. **public String getInitParameter(String name):**Returns the parameter value for the specified parameter name.
2. **public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters.
3. **public void setAttribute(String name, Object object):**sets the given object in the application scope.
4. **public Object getAttribute(String name):**Returns the attribute for the specified name.
5. **public Enumeration getInitParameterNames():**Returns the names of the context's initialization parameters as an Enumeration of String objects.

6. **public void removeAttribute(String name):**Removes the attribute with the given name from the servlet context.

How to configure data in web.xml file and access in servlet

---

if we want to configure data in web.xml file we have to write following tag in web.xml file

```
<context-param>
   <param-name>name of parameter</param-name>
  <param-value>value of parameter</param-value>
</context-param>
```

Example:

```
<context-param>
   <param-name>driver</param-name>
  <param-value>com.mysql.cj.jdbc.Driver</param-value>
</context-param>
```

Once we initialize the data in web.xml file and want to access in servlet using ServletContext we have following steps.

1. add javax.servlet.* and javax.servlet.http.* package in application

---

```
import javax.servlet.*;
import javax.servlet.http.*;
```

2. Create reference of ServletContext

If we want to create reference of ServletContext we have method name as getServletContext() of ServletConfig interface and this method return reference of ServletContext

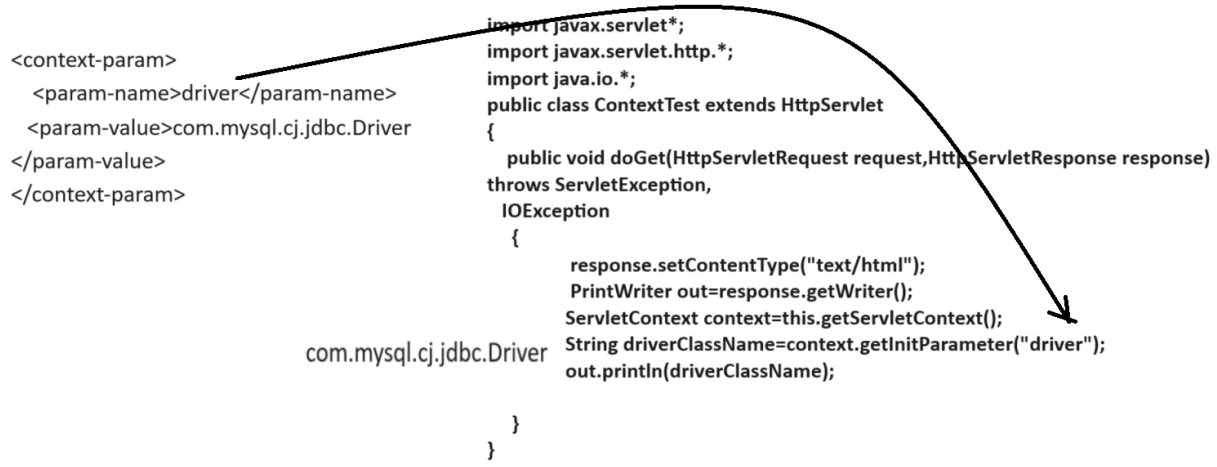
Syntax: ServletContext getServletContext();

```
import javax.servlet*;
import javax.servlet.http.*;
import java.io.*;
public class ContextTest extends HttpServlet
{
   public void doGet(HttpServletRequest request, HttpServletResponse response)throws
ServletException, IOException
   {                 response.setContentType(“text/html”);
                      PrintWriter out=response.getWriter();
```

```
                ServletContext context=this.getServletContext();

    }
}
```

3. Using its getInitParameter() method for access context data.

Once we create object of ServletContext then we can access data using context object configured by web.xml file



Complete Source code

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee" xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" id="WebApp_ID" version="4.0">
 <display-name>ContextConfigApplication</display-name>
  <context-param>
     <param-name>driver</param-name>
     <param-value>com.mysql.cj.jdbc.Driver</param-value>
  </context-param>
</web-app>
```

ContextParamServ.java

```
package org.techhub;

import java.io.IOException;
import java.io.PrintWriter;
```

```
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/contexttest")
public class ContextParamServ extends HttpServlet {

	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		ServletContext context=this.getServletContext();
		String driverClassName=context.getInitParameter("driver");
		out.println("Context parameter is "+driverClassName);
	}
	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

			doGet(request, response);
	}

}
```

ServletConfig

ServletConfig is object created at server side and it is also used for configure data in web.xml file and access in Servlet.

An object of ServletConfig is created by the web container for each servlet. This object can be used to get configuration information from web.xml file.

If the configuration information is modified from the web.xml file, we don't need to change the servlet. So it is easier to manage the web application if any specific content is modified from time to time.

Q. what is diff between ServletContext and ServletConfig?

If we think about ServletContext or if we configure data for ServletContext then data is accessible in all servlets of application or all pages in application and if we think about ServletConfig data is accessible within a specified servlet means ServletConfig is for particular Servlet and ServletContext for all servlets.

Advantage of ServletConfig

The core advantage of ServletConfig is that you don't need to edit the servlet file if information is modified from the web.xml file.

Methods of ServletConfig interface

1. **public String getInitParameter(String name):**Returns the parameter value for the specified parameter name.
2. **public Enumeration getInitParameterNames():**Returns an enumeration of all the initialization parameter names.
3. **public String getServletName():**Returns the name of the servlet.
4. **public ServletContext getServletContext():**Returns an object of ServletContext.

How to get the object of ServletConfig

1. **getServletConfig() method** of Servlet interface returns the object of ServletConfig

How to configure data for ServletConfig

If we want to configure data for ServletConfig we have to use following steps.

Write following tags in web.xml file

```
<servlet>
   <servlet-name>name of servlet</servlet-name>
   <servlet-class>class name of servlet</servlet-class>
         <!—config parameter -->
         <Init-param>
          <param-name> name of parameter</param-name>
          <param-value>value of parameter</param-value>
         </init-param>
</servlet>
<servlet-mapping>
   <servlet-name>name of servlet</servlet-name>
```

```
    <url-pattern>/urlname</url-pattern>
</servlet-mapping>
```

If we think about above configuration we have tag name as init-param this parameter indicate it is parameter for ServletConfig
Once we configure parameter for a servlet config we can access in Servlet

Steps to access config parameter in servlet

---

1. add javax.servlet.* and javax.servlet.http.* package

```
import  javax.servlet.*;
import javax.servlet.http.*;
```

2. Create reference of ServletConfig

If we want to create reference of ServletConfig we have getServletConfig() method of GenericServlet

Syntax: ServletConfig ref= this.getServletConfig();

Sample code

```
import javax.servlet*;
import javax.servlet.http.*;
import java.io.*;
public class ContextTest extends HttpServlet
{
   public void doGet(HttpServletRequest request, HttpServletResponse response)throws
ServletException,
  IOException
    {
                    response.setContentType("text/html");
                    PrintWriter out=response.getWriter();
                    ServletConfig config=this.getServletConfig();

    }
}
```

3. Call its getInitParameter () method

String parameterName(String paramName) : this method accept parameter using its name.

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee" xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd" id="WebApp_ID" version="4.0">
 <display-name>ContextConfigApplication</display-name>
  <context-param>
     <param-name>driver</param-name>
     <param-value>com.mysql.cj.jdbc.Driver</param-value>
  </context-param>

  <servlet>
   <servlet-name>f</servlet-name>
   <servlet-class>org.techhub.First</servlet-class>
    <init-param>
      <param-name>url</param-name>
      <param-value>jdbc:mysql://localhost:3306/mysql</param-value>
    </init-param>
  </servlet>
  <servlet-mapping>
    <servlet-name>f</servlet-name>
    <url-pattern>/first</url-pattern>
  </servlet-mapping>

  <servlet>
   <servlet-name>s</servlet-name>
   <servlet-class>org.techhub.Second</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>s</servlet-name>
    <url-pattern>/second</url-pattern>
  </servlet-mapping>

</web-app>
```

First.java

```
package org.techhub;

import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletConfig;
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
```

```
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class First extends HttpServlet {

	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		ServletContext context=this.getServletContext();
		String dvalue=context.getInitParameter("driver");
		out.println("Context Data is "+dvalue+"<br>");
		ServletConfig config=this.getServletConfig();
	   String cvalue=config.getInitParameter("url");
	   out.println("Config Data is "+cvalue);
	   out.println("<a href='second'>Second Servlet</a>");
	}

	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

		doGet(request, response);
	}

}
```

Second.java

```
package org.techhub;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletConfig;
import javax.servlet.ServletContext;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class Second extends HttpServlet {
```

```
	protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		response.setContentType("text/html");
		PrintWriter out=response.getWriter();
		ServletContext context=this.getServletContext();
		String dvalue=context.getInitParameter("driver");
		out.println("Context Data is "+dvalue+"<br>");
		ServletConfig config=this.getServletConfig();
	   String cvalue=config.getInitParameter("url");
	   out.println("Config Data is "+cvalue);
	   out.println("<a href='first'>First Servlet</a>");

	}
	protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
		doGet(request, response);
	}
}
```

Interview Question on Servlet

Q. what is servlet?

Servlet is java program or application which can execute within a server and it help us to develop the dynamic web pages and accept the request from web clients and send response to web client as per their request and servlet execute by using http protocol because it is used for develop the web applications using java.

Q. Why use Servlet or what is benefit of Servlet?

---

1. Design dynamic web pages at server side
2. Accept Request from web client and send response
3. Provide multi-threading feature for manage multiple client request in background
4. It is used for writing business logic layer in application
5. Generate dynamic html at run time

Etc.

Q. Where Servlet-api or packages is present?

All Servlet packages present in servlet-api.jar file and this file present in tomcat\lib folder

Q. what is use of web.xml file or what is role of deploy descriptor xml file in servlet?

---

web.xml is configuration file of servlet which is used for configure the servlet url for access end user , as well as it is used for configure the context data ,config data and this file provide detail information about servlet to servlet container or tomcat container for create instance of or object of servlet class internally

Example:

```
<servlet>
  <servlet-name>s</servlet-name>
  <servlet-class>org.techhub.Second</servlet-class>
</servlet>
<servlet-mapping>
   <servlet-name>s</servlet-name>
   <url-pattern>/second</url-pattern>
</servlet-mapping>
```

You can configure servlet in web.xml file mention in above tags

Q. How to submit HTML form to servlet?

---

if we want to submit form to servlet we have form tag in HTML
Syntax:
<form name='formname' action='urlname' method='GET/POST'>
</form>

Q. what is diff between doGet() and doPost() method?

---

1. In the case doGet() all request parameter appended in URL with request header information means parameter name and parameter data but in doPost() data not append in URL send by header of the page

Url using doGet: http://localhost:port/projectname/urlname?name=value&name=value
URL using doPost: http://localhost:port/projectname/urlanme

2. In the case of doGet() most server not allow huge information in URL so doGet() cannot send large request data to server and it is not happen in the case of doPost

3. doGet() is not secure so normally doGet() not recommended to send sensitive requested to server then doPost() mostly preferable

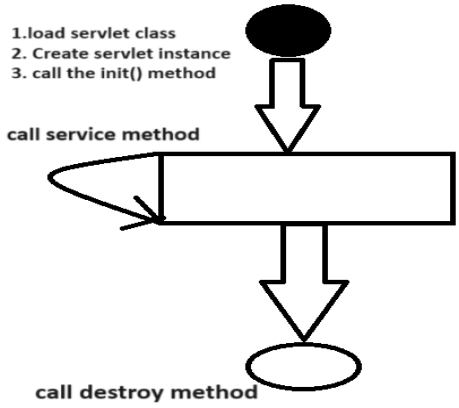
4. doGet() normally recommended in the case of URL Rewriting or when user want to send data via URL from one page to another page and doPost() normally recommended when we want to submit form or when we want to upload file.

Q. Explain life cycle servlet?

Life cycles means decide how servlet instance created and destroy whole process of servlet execution called as life cycle of servlet.

Steps in servlet life cycle

1. Servlet class is loaded
2. Servlet instance is created
3. init method is invoked
4. service method is invoked
5. destroy method is invoked.



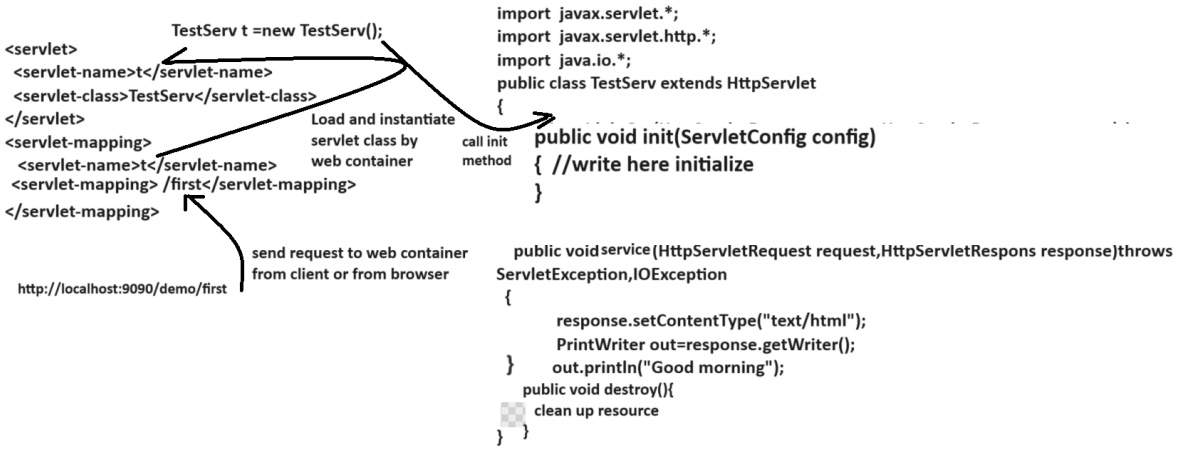
1. Servlet class is loaded: the class loader is responsible to load the servlet class. The servlet class is loaded when the first request for the servlet is received by the web container.

2. Instantiate object of servlet class: The web container creates the instance of servlet after load the servlet class then servlet instance is created only once in the servlet life cycle.

3. Call Init method: The web container calls the init method only once after creating the servlet instance. The init method is used to initialize the servlet. it is the life cycle method of the javax.servlet.Servlet interface means here we can write here logic which we want to execute after creating servlet object.

4. Service method is invoked: service method work as background method means servlet container call service method every time on every request of client and accept the request by using ServletRequest interface and process on request and provide the response to client using ServletResponse interface in the form of text/html by default.

5. Destroy method: destroy method call by servlet container before remove the instance of servlet from web container means using destroy we can clean the resource before remove instance of servlet.
Example: memory, thread, connection etc.
Syntax: void destroy ()
Following diagram show the execution of servlet life cycle



Q. what is diff between GenericServlet and HttpServlet?

GenericServlet	**HttpServlet**
Generic servlet is protocol independent means generic servlet can work with any protocol	HttpServlet is protocol dependent means HttpServlet can work with only http protocol.
Generic servlet is a member of javax.servlet. package	HttpServlet is a member of javax.servlet.http.* package
Generic servlet is parent of HttpServlet and implementer class of Servlet interface internally	HttpServlet is child of GenericServlet and acquire life cycle method of Servlet interface form Genericservlet
GenericServlet not support to session management	HttpServlet support to session management
Methods of GenericServlet 1. ServetContext getServletContext() 2. ServletConfig getServletConfig() 3. String getInitParameter(String paramName) 4. Enumeration getInitParameterNames()	**Methods of HttpServlet** void doGet(HttpServletRequest, HttpServletResponse) doPost() doDelete(), doHead(), doPut() etc.

Q. Explain the internal Hierarchy of Servlet?

```
  interface Servlet{
        public void init(ServletConfig config){
  }
 public void service(ServletRequest request, ServletResponse response)
  public void destroy()
}
abstract class GenericServlet implements Servlet{
          ServletContext getServletContext()
          ServletConfig getServletConfig()
          String getInitParameter(String paramName)
```

```
        String getServletInfo();
}
abstract class HttpServlet extends GenericServlet
{   public void doGet(HttpServletRequest,HttpServletResponse)throws ServletException,IOException
   public void doPost(HttpServletRequest,HttpServletResponse)throws ServletException,IOException
   public void doHead(HttpServletRequest,HttpServletResponse)throws ServletException,IOException
   public void doDelete(HttpServletRequest,HttpServletResponse)throws ServletException,IOException
   public void doPut(HttpServletRequest,HttpServletResponse)throws ServletException,IOException
}
```

Q. what is session and why session in application or benefits of session?

Note: refer session points from above notes.

Q. what is cookies and why use it?
Note: refer cookie points from above notes

Q. what is diff between cookie and HttpSession?

Cookie and HttpSession both are ways to manage the session in web application with some differences.

HttpSession	**Cookie**
HttpSession is interface from javax.servlet.http.*	Cookie is class from javax.servlet.http.* package
HttpSession reference is created by getSession() method of HttpServletRequest. HttpSession session=request.getSession(boolean)	Cookie need to create object and pass two parameter in it Cookie c= new Cookie(String name, String value)
HttpSession delete its data when we close the browser means Httpsession object life present whenever browser in running	Cookie can hold session data for longer time period after browser close if we set age of cookie means persistence cookie
HttpSession object created at server side and store its data at server server just send session id to client for identify client for next request	Cookie Created at server side and send its data as response to client means cookie data stored in browser
HttpSession always work like as temporary	Cookie has two types Persistence cookie and temporary cookie and cookie is responsible for leak the user data so try to avoid store important data in cookie object
Methods of HttpSession Void setAttribute(Object key, Object value) Object getAttribute(Object key): Void invalidate(): destroy the session Boolean isNew(): check session is new or not Etc.	Methods of Cookie String getName() String getValue() Void setMaxAge(int seconds) Etc.

Q. What is servlet context and why use it?
Note: refer topic mention in above notes
Q. what is SerlvetConfig and why use it?
Note: refer topic mention in above notes

Q. what is diff between SerlvetContext and ServletConfig?

---

1. SerlvetContext work as global object in application means if we configure data in ServletContext then it is accessible by all servlets in application and ServletConfig is local for every servlet means config data configure for particular servlet and accessible within a specified servlet
2. You can create reference of ServletContext like as ServletContext getServletContext() and you can create ServletConfig object like as ServletConfig getServletConfig()
Example:

```
ServletContext context=this.getServletContext()
```

Example:

```
ServletConfig config=this.getServletConfig()
```

3. You can Configure parameter for context object like as in web.xml

```
<context-param>
  <param-name>driver</param-name>
 <param-value>com.mysql.cj.jdbc.Driver</param>
</context-param>
```

You can configure parameter for config object like as

```
 <servlet>
  <servlet-name>f</servlet-name>
  <servlet-class>org.techhub.First</servlet-class>
 <init-param>
   <param-name>url</param-name>
   <param-value>jdbc:mysql://localhost:3306/mysql</param-value>
 </init-param>
</servlet>
```

You can access both data using getInitParameter() method
Example:

```
ServletContext context=this.getServletContext();
String driver=context.getInitParameter("driver");

ServletConfig config=this.getServletConfig();
String url=config.getInitParameter("url");
```

Q. What is diff between HttpSession and ServletContext?
1. HttpSession is object created at server side for every browser or for every client and ServletContext is object created for every project and it is known as application object

2. HttpSession data is separate for every client but context data or application object data is common for all clients
3. HttpSession life present whenever client is running or whenever browser is running and ServletContext data life present whenever server is running.
4. Using context object we can configure data in web.xml and access in servlet but it is not happen with HttpSession
5. HttpSession help us to provide data privacy and Context help to provide generalize data in all clients
6. HttpSession can create by using
HttpSession ref=request.getSession(boolean)
and Context object can create using
ServletContext context= this.getServletContext():

RequestDispatcher Interface	sendRedirect method
Forwards a request within the same application	Redirects a request to a different application or URL
Does not change the URL in the browser	Changes the URL in the browser.
Can use the forward and include methods	Uses the sendRedirect method on the HttpServletResponse object
Used for internal request forwarding	Used for external request redirection
request.getRequestDispatcher (“/servlet2”).forward(request,response);	response. sendRedirect(“https://www.example.com“);
request.getRequestDispatcher (“/servlet2”). include(request,response);	response.sendRedirect(“/servlet2”);